

Spectral Frequency-Unit Utilities Specification

Canonical THz Axis and Derived Angular-Frequency, Wavenumber, and Energy Coordinates

mdstats

2026-07-14

Purpose

This document specifies the private module

`mdstats/analysis/_spectral_units.py`

The module converts the canonical spectral frequency axis into equivalent coordinates used in molecular-dynamics and spectroscopy plots. It performs unit conversion only; it never interpolates, resamples, or renormalizes a spectrum.

Canonical convention

The canonical frequency is cycles per picosecond:

$$f[\text{ps}^{-1}].$$

Numerically,

$$1\text{ps}^{-1} = 1\text{THz}.$$

All derived axes refer to the same bins.

Public status

The module is private. Public result objects expose the converted arrays, but users do not need to call this helper directly.

Function

```
def convert_frequency_axes(  
    frequencies_thz: ArrayLike,  
) -> tuple[FloatArray, FloatArray, FloatArray]:  
    ...
```

The returned arrays are:

1. angular frequency in rad/ps;
2. spectroscopic wavenumber in cm^{-1} ;
3. quantum energy hf in meV.

Mathematical conversions

Angular frequency:

$$\omega = 2\pi f.$$

Wavenumber:

$$\tilde{\nu} = \frac{f}{c}.$$

Because SciPy stores c in m/s and the input is THz,

$$\tilde{\nu}[\text{cm}^{-1}] = \frac{f_{\text{THz}} 10^{12}}{100c}.$$

Energy:

$$E = hf,$$

or

$$E[\text{meV}] = f_{\text{THz}} \frac{h10^{15}}{e},$$

where e is the joule value of one electron volt.

Inputs and constraints

- the input is one-dimensional and nonempty;
- all values are finite and nonnegative;
- ordering and uniformity are not changed;
- zero is allowed and maps to zero on every axis.

Numerical source

The physical constants are read from `scipy.constants`. These conversions are standard dimensional relations, not a borrowed specialized algorithm. SciPy is the direct software source [1].

Tests

`tests/test_spectral.py` verifies:

- exact 2π angular conversion;
- the known values $1\text{THz} \approx 33.3564\text{cm}^{-1}$ and $1\text{THz} \approx 4.13567\text{meV}$;
- rejection of multidimensional, negative, empty, and nonfinite inputs.

Reference

[1] P. Virtanen, R. Gommers, T. E. Oliphant, et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.